GEMA MX

Genomic Analysis of Mammary cancer in Mexico

Reporte de Arquitectura e Implementación del Pipeline

Pipeline de Oncología de Precisión · TCGA-BRCA · Versión 1.0

Líder del Proyecto	Ing. Ximena Gutiérrez Gálvez
Asesores	Dr. Julio Waissman Vilanova Mtro. Héctor Gutiérrez Ibarra
Institución	Universidad de Sonora (UNISON)
Fecha	Marzo 2026

1. Resumen Ejecutivo

GEMA MX (Genomic Analysis of Mammary cancer in Mexico) es un pipeline de oncología de precisión que integra ingesta automatizada de datos genómicos, predicción estructural de proteínas con inteligencia artificial y simulación de docking molecular. Su objetivo es reclasificar variantes genéticas de Significado Incierto (VUS) en cáncer de mama, identificando candidatos farmacológicos para repositionamiento terapéutico.

El pipeline procesa 1,084 casos de cáncer de mama del proyecto TCGA-BRCA, prioriza variantes missense en 15 genes clave, predice estructuras 3D de proteínas mutadas con Boltz-1/ESMFold, y ejecuta simulaciones de docking molecular con 11 fármacos usando AutoDock Vina 1.2.3. Los resultados se visualizan en una aplicación web estática con visualizador molecular 3D integrado (PDBe Molstar).

2. Arquitectura General del Sistema

2.1 Diagrama de Componentes

Módulo	Tecnología	Salida
M1 — Ingesta	Python · cBioPortal API / GDC API · MongoDB · Pandas	raw_mutations.json top_variants.json mutation_stats.json
M2 — FASTA	Python · UniProt REST API · Biopython · MutableSeq	data/fasta/{gen}_WT.fasta data/fasta/{gen}_{cambio}.fasta
M3 — Estructuras 3D	Boltz-1 (Colab T4 GPU) ESMFold API OpenBabel 3.1.1	data/structures/boltz_results_*/ predictions/*.cif data/structures/*.pdb
M4 — Docking	AutoDock Vina 1.2.3 RDKit · Meeko · ChEMBL API OpenBabel (receptor PDBQT)	data/docking/vina_affinities.json data/docking/results/*.pdbqt structural_metrics.json
M5 — Web App	HTML5 · CSS3 · JS Vanilla Plotly.js · PDBe Molstar v3.2 Antigravity (framework)	index.html dashboard.js · app.js analysis.js · structural.js docking.js · report.js

2.2 Flujo de Datos

El flujo es completamente lineal y unidireccional. Cada módulo consume los archivos JSON producidos por el anterior:

- cBioPortal API → ingest_data.py → raw_mutations.json + top_variants.json
- top_variants.json → fetch_fasta.py → data/fasta/*.fasta
- data/fasta/*.fasta → generate_structures.py → data/structures/*.pdb/.cif
- data/structures/ → prepare_docking.py + run_docking.py → vina_affinities.json
- Todos los JSON → app web estática → visualización en navegador

2.3 Base de Datos — MongoDB

MongoDB se utiliza exclusivamente durante la fase de ingesta y procesamiento (M1). Corre localmente en localhost:27017 y almacena tres colecciones:

Colección	Contenido
mutations	Todas las mutaciones missense filtradas de los 15 genes clave. Campos: hugo_symbol, protein_change, variant_classification, sample_id, chromosome, start_position.
top_variants	Ranking Top-20 variantes por frecuencia. Agrega: frequency, cohort_percentage, classification, rank.
fasta_sequences	Secuencias WT y mutadas por variante, con su uniprot_id, mutation_pos y ambas cadenas de aminoácidos.

Importante: MongoDB no se usa en tiempo de ejecución de la app web. Es un almacén intermedio de la fase científica. La app consume únicamente archivos JSON estáticos.

3. Módulo M1 — Ingesta y Priorización de Variantes

3.1 Fuente de Datos

El pipeline utiliza la API de cBioPortal (www.cbioportal.org/api), que expone datos del proyecto TCGA-BRCA estandarizados. El estudio utilizado es brca_tcga con perfil molecular brca_tcga_mutations.

En paralelo, el notebook de Jupyter también referencia la API directa del GDC (Genomic Data Commons del NCI/NIH) mediante el endpoint https://api.gdc.cancer.gov/ssm_occurrences para descarga con filtros JSON complejos.

3.2 Genes Priorizados

El pipeline filtra 15 genes de interés clínico divididos en tres niveles de relevancia:

Tier	Genes	Criterio
Tier 1	PIK3CA, TP53, ERBB2, BRCA1, BRCA2	Genes accionables con terapia dirigida disponible
Tier 2	CDH1, GATA3, MAP3K1, KMT2C, PTEN	Drivers frecuentes en TCGA-BRCA
Tier 3	RB1, ARID1A, SF3B1, AKT1, CBFB	Relevantes en estudios TCGA previos

3.3 Funciones Principales de ingest_data.py

fetch_mutations()

Descarga todas las mutaciones del estudio en bloques paginados de 500 muestras. Primero obtiene la lista de todos los sample IDs del estudio, luego ejecuta solicitudes POST paginadas al endpoint /mutations/fetch con cada bloque:

```
POST /molecular-profiles/brca_tcga_mutations/mutations/fetch
Body: { sampleIds: [chunk_de_500_ids] }
```

Retorna una lista de diccionarios con todos los campos de cada mutación. Total descargado: ~33,000 mutaciones del cohort completo.

get_gene_mapping(entrez_ids)

Mapea identificadores Entrez Gene ID (numéricos) a símbolos HUGO (PIK3CA, TP53, etc.) usando el endpoint /genes/fetch. Procesa hasta 500 IDs por solicitud POST.

process_and_store(data, gene_map, total_samples)

Esta es la función central del módulo M1. Realiza cuatro operaciones en secuencia:

- Mapeo de columnas: renombra mutationType → variant_classification, proteinChange → protein_change, sampleId → sample_id.
- Filtrado: selecciona únicamente mutaciones Missense en los 15 genes clave.
- Ranking: agrupa por (gen, cambio_proteico), cuenta frecuencia, calcula cohort_percentage y asigna clasificación clínica.

- Exportación: genera raw_mutations.json (cohort completo), top_variants.json (Top-20 rankeados) y mutation_stats.json (distribuciones para gráficas).

3.4 Esquema de Clasificación

La clasificación de variantes se asigna automáticamente con base en frecuencia en la cohorte:

Clasificación	Umbral	Significado
Patogénica	>= 5% cohorte	Alta frecuencia, evidencia de driver oncogénico
Prob. Patogénica	1 - 4.9%	Frecuencia media, posible driver
VUS	0.5 - 0.99%	Variante de Significado Incierto — objetivo del pipeline
Baja Frecuencia	< 0.5%	Probablemente pasajera o rara

4. Módulo M2 — Generación de Secuencias FASTA

4.1 Objetivo

Este módulo construye las secuencias de aminoácidos de cada proteína en su forma Wild-Type (WT) y en su forma mutada, para ser usadas como entrada en la predicción estructural de M3.

4.2 Integración con UniProt

El script fetch_fasta.py utiliza la API REST de UniProt para obtener las secuencias canónicas en formato FASTA:

```
GET https://rest.uniprot.org/uniprotkb/search
   ?query=gene:{GENE} AND organism_id:9606 AND reviewed:true
   &format=json&size=1
```

El parámetro reviewed:true asegura que solo se use la secuencia de la base de datos Swiss-Prot (curada manualmente), no TrEMBL (predicha). Luego descarga el FASTA completo con:

```
GET https://www.uniprot.org/uniprot/{accession}.fasta
```

4.3 Función apply_mutation()

Recibe la secuencia WT en formato FASTA y el código de cambio proteico (ej. H1047R) y produce la secuencia mutada:

- Parsea el cambio con regex: r'([A-Z])(\d+)([A-Z])' → aminoácido_WT, posición, aminoácido_MUT
- Convierte la secuencia a MutableSeq de Biopython
- Verifica que el aminoácido en la posición indicada coincida con el WT esperado (validación de consistencia)
- Realiza la sustitución puntual en la posición 1-indexada
- Retorna ambas secuencias como strings: (wt_str, mut_str, pos)

Las secuencias se guardan en disco y en MongoDB:

```
data/fasta/{gen}_WT.fasta     <- secuencia original
data/fasta/{gen}_{cambio}.fasta  <- secuencia con mutación puntual
```

4.4 Gestión de Proteínas Largas — generate_structures.py

Dado que ESMFold tiene un límite de ~390 aminoácidos, las proteínas grandes requieren un fragmento de dominio:

```
def get_domain_fragment(seq, mut_pos, window=180):
    start = max(0, mut_pos - window - 1)
    end   = min(len(seq), mut_pos + window)
    return seq[start:end], start + 1
```

Esto extrae una ventana de 360 aminoácidos centrada en la posición de la mutación. Proteínas como PIK3CA (1,068 aa), SF3B1 (1,304 aa) y PTEN (403 aa) se procesan en fragmentos, preservando el contexto estructural de la mutación.

5. Módulo M3 — Predicción Estructural 3D

5.1 Herramientas Utilizadas

Herramienta	Tipo	Uso en el Pipeline
Boltz-1	Modelo IA (GPU)	Predicción principal. Se ejecuta en Google Colab con GPU T4. Produce archivos CIF (mmCIF) con scores pLDDT por residuo.
ESMFold API	API HTTP	Predicción complementaria vía API pública (esmatlas.com). Acepta secuencias de hasta ~390 aa. Produce archivos PDB.
PDBe Molstar v3.2	Visualizador Web	Renderizado 3D interactivo en la app web. Lee archivos CIF y PDB directamente desde disco.

5.2 Integración con Boltz-1 en Google Colab

El notebook 01_tcga_exploration.ipynb orquesta la ejecución de Boltz-1. La configuración del entorno resuelve tres incompatibilidades críticas:

- NumPy y Pandas: se fija numpy==1.26.4 y pandas==2.2.2 para evitar errores de incompatibilidad binaria con extensiones C.
- PyTorch Lightning: se fija pytorch-lightning==2.1.4 compatible con la API interna que Boltz-1 utiliza (la versión 2.2+ rompe el entrenamiento).
- Google Drive: se montan symlinks /content/data → Drive/GEMAMX/data para persistencia entre sesiones de Colab.

El comando de predicción que ejecuta el notebook por cada variante es:

```
boltz predict data/fasta/{variante}.fasta \
    --out_dir data/structures/boltz_results_{variante} \
    --use_msa_server \
    --recycling_steps 3
```

5.3 Estructura de Salida de Boltz-1

Boltz-1 produce una carpeta por variante con la siguiente estructura:

```
data/structures/boltz_results_{VARIANTE}/
  predictions/
    {VARIANTE}/
      {VARIANTE}_model_0.cif      <- estructura 3D en formato mmCIF
      confidence_{VARIANTE}_model_0.json  <- scores pLDDT
```

El archivo confidence JSON contiene el campo complex_plddt (valor 0-1) que se convierte a escala 0-100 para mostrarlo en la interfaz web.

5.4 scan_boltz.py — Gestión del Manifest

Este script escanea todos los directorios de salida y construye manifest_structures.json, que es el índice central que la app web usa para saber qué estructuras están disponibles:

```
glob.glob('data/structures/**/predictions/**/*.cif', recursive=True)
```

Por cada archivo encontrado extrae el nombre de la variante, el path relativo y el pLDDT del JSON de confianza. Construye el manifest como:

```
{"PIK3CA_H1047R": "data/structures/boltz_results_PIK3CA_H1047R/...cif",
 "TP53_R248Q":    "data/structures/boltz_results_TP53_R248Q/...cif", ...}
```

La app web de structural.js lee este manifest al inicializar para cargar las estructuras en PDBe Molstar.

6. Módulo M4 — Docking Molecular con AutoDock Vina

6.1 Descripción del Proceso

El docking molecular simula la interacción física entre una proteína (receptor) y un fármaco (ligando) para predecir la pose de unión y calcular la energía libre de unión en kcal/mol. Valores más negativos indican mayor afinidad de unión.

6.2 Catálogo de Fármacos

Se evalúan 11 fármacos obtenidos de la base de datos ChEMBL, todos inhibidores de vías de señalización relevantes en cáncer de mama:

Fármaco	ChEMBL ID	Target	Estado
Alpelisib	CHEMBL3301610	PI3Kα	FDA aprobado
Abemaciclib	CHEMBL3545110	CDK4/6	FDA aprobado
Palbociclib	CHEMBL1771552	CDK4/6	FDA aprobado
Ribociclib	CHEMBL3545283	CDK4/6	FDA aprobado
Capivasertib	CHEMBL3301612	AKT1/2/3	FDA aprobado
Everolimus	CHEMBL1560600	mTOR	FDA aprobado
Gedatolisib	CHEMBL3188482	PI3K/mTOR	Ensayo clínico
Taselisib	CHEMBL3301609	PI3Kα	Ensayo clínico
Pictilisib	CHEMBL2178688	PI3K	Ensayo clínico
Buparlisib	CHEMBL2180717	Pan-PI3K	Ensayo clínico
Copanlisib	CHEMBL3137587	Pan-PI3K	FDA aprobado

6.3 Pipeline Paso a Paso — prepare_docking.py

Paso 1 — Descarga de SDF desde ChEMBL

Para cada fármaco en el catálogo se descarga el archivo SDF 2D desde la API de ChEMBL:

```
GET https://www.ebi.ac.uk/chembl/api/data/molecule/{chembl_id}.sdf
```

El SDF contiene la estructura molecular en 2D con todas las propiedades del compuesto.

Paso 2 — Conversión SDF → PDBQT (Ligando)

Los archivos SDF 2D se convierten a formato PDBQT 3D usando la librería RDKit junto con Meeko:

- RDKit lee el SDF y selecciona el fragmento más grande (elimina contraiones y sales).
- AllChem.AddHs() agrega hidrógenos explícitos a la molécula.
- AllChem.EmbedMolecule(mol, ETKDG()) genera las coordenadas 3D usando el algoritmo ETKDG.

- AllChem.MMFFOptimizeMolecule(mol) optimiza la geometría con el campo de fuerzas MMFF.
- MoleculePreparation de Meeko agrega los tipos de átomo AutoDock4 (AD4), cargas parciales y gestiona las ramas rotables del ligando.
- El resultado se escribe como archivo .pdbqt, formato nativo de AutoDock Vina.

Paso 3 — Preparación del Receptor PDBQT

Los archivos CIF/PDB de las proteínas predichas se convierten a PDBQT para que Vina los procese como receptores rígidos:

- OpenBabel 3.1.1 limpia el PDB: elimina heteroátomos, agrega hidrógenos a pH 7.4 y produce un PDB estándar.
- La función pdb_to_pdbqt_simple() realiza una conversión 'quirúrgica' leyendo cada línea ATOM/HETATM y construyendo el PDBQT con el formato exacto de columnas que Vina requiere.
- Para cada átomo se asigna el tipo AutoDock4 (AD4 type) basado en el símbolo del elemento: H→HD, N→NA, O→OA, etc.

Paso 4 — Cálculo del Centro del Sitio Activo

En lugar de centrar la caja de búsqueda en el centro de masa global de la proteína (que podría estar lejos del sitio activo), el pipeline localiza específicamente el residuo mutado:

```
def get_mutation_center(pdb_path, variant_id):
    # Extrae posición del ID: PIK3CA_H1047R -> 1047
    # Promedia coordenadas XYZ de todos los átomos de ese residuo
    # Retorna (cx, cy, cz) como centro de la caja de búsqueda
```

Esto concentra el docking en el sitio de la mutación, que es donde los cambios conformacionales son más relevantes para la afinidad del fármaco.

Paso 5 — Ejecución de AutoDock Vina 1.2.3

Para cada par receptor-ligando se ejecuta Vina con los siguientes parámetros:

Parámetro	Valor	Descripción
--exhaustiveness	8	Nivel de exploración del espacio conformacional. 8 es el valor estándar de producción.
--num_modes	5	Número de poses de unión a generar por par.
--size_x/y/z	25 Å	Dimensiones de la caja de búsqueda centrada en el sitio de mutación.
--out	{variante}_{fármaco}_out.pdbqt	Archivo PDBQT con las 5 mejores poses de unión.
--log	{variante}_{fármaco}_log.txt	Log con tabla de afinidades para todas las poses.

Paso 6 — Parseo de Resultados

La función parse_affinity() extrae el mejor score (pose 1) del log de Vina usando expresiones regulares robustas que soportan tanto el formato de Vina 1.1.x como 1.2.x:

```
# Vina 1.2.x: '   1       -8.903       0.000       0.000'
re.search(r'^\s*1\s+([-+]?\d+\.\d+)', content, re.MULTILINE)
```

El resultado es un diccionario anidado de afinidades: { variante: { fármaco: kcal/mol } } guardado en vina_affinities.json.

Paso 7 — Actualización de Métricas

Los scores reales de Vina se integran en structural_metrics.json, que ya contiene los datos estructurales (pLDDT, RMSD). Para cada variante se añade:

- best_vina_drug: nombre del fármaco con mayor afinidad (score más negativo)
- best_vina_score: valor en kcal/mol
- vina_scores: diccionario completo { fármaco: score } para generar rankings en la app

7. Notebook de Jupyter — 01_tcga_exploration.ipynb

7.1 Propósito

El notebook es el orquestador de toda la fase científica computacionalmente intensiva. Está diseñado para ejecutarse en Google Colab con GPU T4 gratuita y persistencia en Google Drive. Es la única parte del pipeline que requiere GPU.

7.2 Estructura del Notebook

Celda	Función
Celda 0 — Setup	Instala dependencias con versiones fijas (numpy 1.26.4, pandas 2.2.2, pytorch-lightning 2.1.4, boltz). Monta Google Drive y crea symlinks. Debe ejecutarse primero y requiere reinicio de entorno.
Celda 0b — Reconexión	Restaura los symlinks y verifica versiones tras cualquier reinicio de Colab. Se ejecuta siempre al volver a abrir la sesión.
Celdas 1-2 — Ingesta GDC	Implementa la descarga directa vía GDC API (alternativa a cBioPortal). Construye los filtros JSON, pagina las descargas y exporta los mismos archivos raw_mutations.json y top_variants.json.
Celdas 3-4 — FASTA	Llama a UniProt REST API para cada variante del Top-20, aplica mutaciones puntuales con Biopython y guarda los archivos .fasta en Drive/GEMAMX/data/fasta/.
Celda 5 — Boltz-1	Ejecuta boltz predict sobre cada archivo FASTA. Itera el Top-20 de variantes. Genera las carpetas boltz_results_{variante}/ con archivos CIF y JSON de confianza.
Celda 6.7 — Docking	Instalada dinámicamente por inject_vina.py. Instala autodock-vina vía apt-get, define rutas de ligandos y receptores, itera todos los pares y guarda vina_affinities.json. Versión simplificada del pipeline para entorno Linux de Colab.
Celda Final — Exportación	Comprime todo el directorio data/ en gemamx_data.zip para descarga. El usuario lo descomprime en D:/ProyectosIA/GEMAMX/data/ para sincronizar la app web local.

7.3 inject_vina.py — Inserción Dinámica de Celdas

Este script utilitario inserta programáticamente el módulo de docking en el notebook sin necesidad de editarlo manualmente. Carga el JSON del notebook, busca el punto de inserción por contenido de celda markdown (buscando '7. Exportación') e inserta una celda markdown y una celda de código antes de ese punto.

Esto permite mantener el notebook base limpio y agregar módulos computacionales complejos de forma controlada por versiones.

8. Aplicación Web — Visualización y Análisis

8.1 Arquitectura Frontend

La app web es completamente estática. No requiere servidor, backend, ni base de datos en tiempo de ejecución. Todo el estado de la aplicación se carga desde archivos JSON locales via fetch() del navegador.

Archivo	Responsabilidad
app.js	Navegación SPA: gestiona qué sección está activa (data-view), carga datos al cambiar de vista, filtros en tiempo real de variantes y dataset base, paginación de raw_mutations.json.
dashboard.js	Inicializa el Dashboard cargando top_variants.json, structural_metrics.json y manifest_structures.json en paralelo. Calcula KPIs: total variantes, hotspot #1, estructuras disponibles, mejor afinidad Vina.
analysis.js	Genera las gráficas de Plotly.js: distribución de frecuencia por gen (Top-20) y distribución de tipos de variante, usando mutation_stats.json.
structural.js	Gestiona los dos visores PDBe Molstar (WT y MUT). Carga estructuras CIF/PDB desde manifest. Implementa: cambio de representación (cartoon/surface/ball-stick), modo científico (coloreado pLDDT), superposición WT/MUT, linked camera sync, RMSD por región.
docking.js	Muestra el ranking de fármacos con scores Vina reales, identifica candidatos repositionables (Δ ≤ -0.5 kcal/mol vs WT), carga la pose de docking ganadora en Molstar.
report.js	Genera el reporte integrado por variante: perfil genómico, métricas estructurales, tabla ordenable de fármacos, gráficas comparativas WT vs MUT, síntesis diagnóstica.

8.2 Visualizador 3D — PDBe Molstar

La visualización molecular 3D usa PDBe Molstar v3.2, una librería de renderizado WebGL de alta calidad desarrollada por el European Bioinformatics Institute (EMBL-EBI). Se integra como plugin JavaScript:

```
<script src="https://cdn.jsdelivr.net/npm/pdbe-molstar@3.2.0/build/pdbe-molstar-plugin.js"></script>
```

Los dos visores (WT y MUT) se inicializan con una estructura de referencia (CBFB_WT.cif) y luego se actualizan con visual.update() al seleccionar una variante. El pipeline de uso es:

- new PDBeMolstarPlugin() crea la instancia del visor
- plugin.render(containerElement, config) inicializa el canvas WebGL
- plugin.visual.update({ customData: { url, format } }) carga nuevas estructuras
- plugin.visual.select({ data: [{ residue_number, color, focus }] }) resalta el sitio de mutación

8.3 Rebuild y Herramientas de Mantenimiento

Script	Función

rebuild_stats.py	Regenera mutation_stats.json desde raw_mutations.json. Recalcula distribuciones de genes y tipos de variante cuando el archivo generado en Colab difiere del local.
scan_boltz.py	Escanea todas las carpetas boltz_results_* y actualiza manifest_structures.json con los CIF disponibles, sus paths y scores pLDDT.
run_docking.py	Versión de re-ejecución limpia del docking con diagnóstico. Detecta automáticamente el ejecutable de Vina disponible, verifica receptores y ligandos, y actualiza los resultados parciales.
inject_vina.py	Inserta dinámicamente la celda de docking en el notebook de Jupyter sin edición manual.

9. Archivos JSON — Contratos de Datos

9.1 top_variants.json

Array de objetos. Cada objeto representa una variante priorizada con sus metadatos completos:

```
[ { "rank": 1, "hugo_symbol": "PIK3CA", "protein_change": "H1047R",
    "frequency": 128, "cohort_percentage": 11.8,
    "chromosome": "3", "start_position": 179234297,
    "classification": "Patog\u00e9nica" }, ... ]
```

9.2 structural_metrics.json

Objeto indexado por variante_ID. Contiene todas las métricas estructurales y de docking integradas:

```
{ "PIK3CA_H1047R": {
    "wt_plddt": "87.3",   "mut_plddt": "85.1",
    "rmsd_global": "1.2",  "rmsd_active": "2.8",
    "rmsd_regions": [{"region": "Dominio Kinasa", "value": 2.8, "color":
"#ef4444"}],
    "best_vina_drug": "Alpelisib", "best_vina_score": -9.2,
    "vina_scores": {"Alpelisib": -9.2, "Palbociclib": -7.8, ...}
  }, ... }
```

9.3 manifest_structures.json

Índice de estructuras disponibles en disco. La app web lo usa para saber qué variantes tienen estructura 3D:

```
{ "PIK3CA_H1047R":
"data/structures/boltz_results_PIK3CA_H1047R/predictions/PIK3CA_H1047R/PIK3CA_H1047
R_model_0.cif",
  "PIK3CA_WT":
"data/structures/boltz_results_PIK3CA_WT/predictions/PIK3CA_WT/PIK3CA_WT_model_0.ci
f",
  "TP53_R248Q":    null, ... }
```

Un valor null indica que la variante está en el ranking pero aún no tiene estructura predicha.

10. Resumen Técnico del Stack

Categoría	Tecnología	Versión / Detalles
Lenguaje	Python	3.12+ con venv
Datos	Pandas	2.2.2
Bioinformática	Biopython	1.83 — SeqIO, MutableSeq
Base de datos	MongoDB	Local 27017 — PyMongo
Predicción IA	Boltz-1	GPU T4 en Google Colab — salida mmCIF
Predicción IA	ESMFold	API pública — proteínas <390 aa — salida PDB
Docking	AutoDock Vina	1.2.3 — exe local en Windows
Quimioinformática	RDKit + Meeko	Generación de PDBQT 3D para ligandos
Conversión	OpenBabel	3.1.1 — preparación de receptores
API Fármacos	ChEMBL	API REST — descarga SDF por ChEMBL ID
API Secuencias	UniProt REST	Swiss-Prot revisadas — FASTA canónico humano
Frontend	HTML5 + JS Vanilla	Sin framework — SPA con data-view pattern
Visualización	PDBe Molstar v3.2	WebGL — CIF/PDB — coloreado pLDDT
Gráficas	Plotly.js 2.32	Distribuciones, comparativas, docking
Entorno Colab	Google Colab + Drive	GPU T4 gratuita + persistencia en Drive